📊 EDA, Data Cleaning & Data Preprocessing

This chapter explains the complete process of preparing data before applying a Machine Learning model.

---

1. What is EDA?

Exploratory Data Analysis (EDA) is the process of examining and understanding a dataset using statistics, summaries, and visualizations.

Simple definition

> **EDA is the process of exploring data to understand its structure, identify patterns, detect problems, and generate useful insights before building a machine learning model.**

EDA helps us answer:

- What data do we have?
- How many rows and columns?
- What type of variables exist?
- Are values missing?
- Are there duplicates?
- Are there outliers?
- Are categories inconsistent?
- Are there relationships between variables?
- What does the target variable look like?

EDA workflow

```
Dataset
   ↓
View Data
   ↓
Statistics
   ↓
Value Counts
   ↓
Missing Value Analysis
   ↓
Visualization
   ↓
Target Exploration
```

```
    ↓
Identify Problems
```

---

2. View the Data

The first step is to look at the dataset.

```
import pandas as pd

df = pd.read_csv("data.csv")
```

View first rows

```
df.head()
```

By default, this displays the first 5 rows.

```
df.head(10)
```

displays the first 10 rows.

View last rows

```
df.tail()
```

View random rows

```
df.sample(5)
```

This is useful because the first few rows may not represent the entire dataset.

---

3. Understand Dataset Structure

Number of rows and columns

```
df.shape
```

Example:

```
(10000, 15)
```

Means:

```
10,000 rows
15 columns
```

Column names

```
df.columns
```

Data types and non-null values

```
df.info()
```

This is one of the most important commands during EDA.

It tells you:

- Column names
- Number of non-null values
- Data types
- Memory usage

---

4. Summary of Statistics

Use:

```
df.describe()
```

It provides statistical information about numerical columns.

Example:

Statistic	Meaning
count	Number of non-missing values
mean	Average
std	Standard deviation
min	Minimum
25%	First quartile
50%	Median
75%	Third quartile
max	Maximum

Example

Suppose:

```
Age
Mean = 35
Median = 32
Min = 18
Max = 90
```

This gives us a quick understanding of the distribution of age.

Include categorical columns

```
df.describe(include="all")
```

---

5. Value Counts

Value counts tells us how frequently each unique value occurs.

```
df["Gender"].value_counts()
```

Example:

```
Female    550
Male      430
Other      20
```

This tells us the distribution of the categories.

Percentage

```
df["Gender"].value_counts(normalize=True) * 100
```

Output:

```
Female    55%
Male      43%
Other      2%
```

Why is this useful?

Value counts help detect:

- Rare categories
- Class imbalance
- Unexpected categories
- Spelling errors
- Inconsistent values

---

6. Missing Value Analysis

Missing values occur when information is unavailable.

Example:

```
Age     Salary     Gender
25      40000      Male
```

```
30      NaN       Female
NaN     50000     Male
```

`NaN` means the value is missing.

Check missing values

```
df.isnull().sum()
```

Missing percentage

```
df.isnull().mean() * 100
```

A useful summary:

```
missing = pd.DataFrame({
    "Missing": df.isnull().sum(),
    "Percentage": df.isnull().mean() * 100
})

missing.sort_values("Percentage", ascending=False)
```

Ways to handle missing values

Remove rows

```
df.dropna()
```

Use carefully because you may lose valuable data.

Fill numerical values

Mean:

```
df["Age"].fillna(df["Age"].mean())
```

Median:

```
df["Age"].fillna(df["Age"].median())
```

Median is often preferable when the data contains outliers or is highly skewed.

Fill categorical values

```
df["Gender"].fillna(df["Gender"].mode()[0])
```

Or use:

```
Unknown
```

when missingness itself may be meaningful.

---

7. Visualization

Visualization helps us understand patterns that may not be obvious from tables.

Main types of visualization

```
Visualization
│
├── Univariate
│
├── Bivariate
│
└── Multivariate
```

A. Univariate Analysis

One variable.

Histogram

Useful for numerical distributions.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.histplot(df["Age"], kde=True)
plt.show()
```

Helps identify:

- Distribution
- Skewness
- Possible outliers
- Concentration of values

Boxplot

```
sns.boxplot(x=df["Salary"])
plt.show()
```

Useful for detecting outliers.

Count plot

For categorical variables:

```
sns.countplot(data=df, x="Gender")
plt.show()
```

---

8. Bivariate Analysis

Bivariate analysis studies the relationship between **two variables**.

Numerical vs Numerical

Use a scatter plot:

```
sns.scatterplot(
    data=df,
    x="Age",
    y="Salary"
)
plt.show()
```

You can investigate whether salary changes as age changes.

Numerical vs Categorical

Example:

```
Gender vs Salary
```

Use:

```
sns.boxplot(
    data=df,
    x="Gender",
    y="Salary"
)
plt.show()
```

Categorical vs Categorical

Example:

```
Gender vs Churn
```

Use:

```
pd.crosstab(df["Gender"], df["Churn"])
```

---

9. Multivariate Visualization

Multivariate analysis looks at several variables simultaneously.

Correlation heatmap

```
corr = df.select_dtypes(include="number").corr()

sns.heatmap(corr, annot=True)
plt.show()
```

This helps identify relationships between numerical variables.

Example:

```
Age ↔ Salary       0.70
Experience ↔ Salary 0.85
Age ↔ Experience   0.75
```

---

10. Target Variable Exploration

The **target variable** is the variable that the ML model is trying to predict.

Example:

```
Features:
Age
Salary
Tenure
Contract

Target:
Churn
```

Classification Target

Suppose:

```
df["Churn"].value_counts()
```

returns:

```
No     7000
Yes    3000
```

This tells us the target distribution.

Check percentages:

```
df["Churn"].value_counts(normalize=True) * 100
```

If the distribution is:

```
No      95%
Yes      5%
```

we have **class imbalance**.

This matters because accuracy can become misleading.

Regression Target

For a continuous target such as house price:

```
df["Price"].describe()
```

Then visualize:

```
sns.histplot(df["Price"], kde=True)
plt.show()
```

Look for:

- Skewness
- Extreme values
- Unusual distributions

---

11. Data Cleaning

Data cleaning means identifying and correcting errors or inconsistencies in a dataset.

Common problems include:

```
Missing values
Duplicates
Wrong data types
Inconsistent categories
Outliers
Logic errors
Domain errors
```

---

12. Removal of Duplicate Data

Duplicates are repeated records.

Check:

```
df.duplicated().sum()
```

View them:

```
df[df.duplicated()]
```

Remove them:

```
df = df.drop_duplicates()
```

Important

Do not automatically delete every duplicate.

For example, in a transaction dataset:

```
Customer A
₹500
10:00 AM

Customer A
₹500
10:00 AM
```

These might be two genuine transactions.

Always understand the dataset before removing duplicates.

---

13. Fix Data Types

Sometimes a column has the wrong data type.

Example:

```
Age
"20"
"25"
"30"
```

It may be stored as `object` instead of integer.

Convert:

```
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
```

Date conversion

```
df["Date"] = pd.to_datetime(df["Date"])
```

Category conversion

```
df["Gender"] = df["Gender"].astype("category")
```

Correct data types are important because ML algorithms expect data in appropriate formats.

---

14. Handle Inconsistent Categories

Categorical data often contains inconsistencies.

Example:

```
Male
male
MALE
M
```

These may represent the same category.

Similarly:

```
Delhi
delhi
```

```
DELHI
New Delhi
```

may require standardization depending on the meaning of the data.

Convert text to lowercase

```
df["Gender"] = df["Gender"].str.lower()
```

Remove unnecessary spaces

```
df["Gender"] = df["Gender"].str.strip()
```

Replace values

```
df["Gender"] = df["Gender"].replace({
    "m": "male",
    "M": "male"
})
```

After cleaning:

```
male
female
other
```

Important

Don't merge categories just because they look similar.

For example:

```
Delhi
New Delhi
```

may or may not represent the same category depending on your dataset and business context.

---

15. Detect and Handle Outliers

An **outlier** is an observation that is unusually different from the majority of the data.

Example:

```
Salary:

30k
35k
40k
42k
45k
48k
500k
```

`500k` may be an outlier.

Method 1 — Boxplot

```
sns.boxplot(x=df["Salary"])
plt.show()
```

Method 2 — IQR

Calculate:

```
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)

IQR = Q3 - Q1
```

Lower and upper boundaries:

```
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

Find outliers:

```
outliers = df[
    (df["Salary"] < lower) |
    (df["Salary"] > upper)
]
```

What should we do with outliers?

There are several possibilities:

```
Outlier
  ↓
Investigate
  ↓
Is it an error?
  ├── Yes → Correct/remove
  │
  └── No
       ↓
  Keep / transform / cap
```

Never follow this rule:

> "Outlier = Delete."

An outlier can be a genuine and important observation.

---

16. Fix Logic Errors

A **logic error** occurs when a value violates a logical relationship in the data.

Example:

```
Age = 25
Number of children = 30
```

This might be impossible depending on the context.

Another example:

```
Start Date = 2025-01-01
End Date = 2024-01-01
```

This may indicate an error.

Strategy

Define logical rules:

```
df[df["End_Date"] < df["Start_Date"]]
```

Then investigate and correct the records.

---

17. Fix Domain Errors

A **domain error** occurs when a value falls outside the valid range for that variable.

Examples:

```
Age = -5
```

Invalid.

```
Percentage = 150%
```

Invalid if the variable is restricted to 0–100%.

```
Rating = 8
```

Invalid if the rating scale is 1–5.

Strategy

Set domain rules:

```
df[(df["Age"] < 0) | (df["Age"] > 120)]
```

Then:

- Correct if possible
- Replace with missing
- Remove if appropriate

---

18. Data Preprocessing

After EDA and cleaning, we prepare the data so that machine learning algorithms can use it.

Definition

> **Data preprocessing is the process of transforming raw, cleaned data into a suitable format for machine learning algorithms.**

The pipeline can look like:

```
Raw Data
   ↓
Cleaning
   ↓
Missing Values
   ↓
Encoding
   ↓
Transformation
   ↓
Scaling
   ↓
Feature Engineering
   ↓
Feature Selection
   ↓
ML Model
```

---

19. Encoding Categorical Variables

Most ML algorithms require numerical input.

Suppose:

```
Gender
-------
Male
Female
Male
Female
```

We need to convert categories into numbers.

A. Label Encoding

Example:

```
Male   → 0
Female → 1
```

Useful when categories are binary or have meaningful ordinal relationships.

B. One-Hot Encoding

Suppose:

```
City
----
Delhi
Mumbai
Pune
```

One-hot encoding creates:

```
Delhi   Mumbai   Pune
1       0        0
0       1        0
0       0        1
```

Using pandas:

```
pd.get_dummies(df, columns=["City"])
```

Or using scikit-learn:

```
from sklearn.preprocessing import OneHotEncoder
```

Important

For nominal categories such as cities, one-hot encoding is generally safer than assigning arbitrary numbers like:

```
Delhi = 1
Mumbai = 2
Pune = 3
```

because those numbers could incorrectly suggest an order.

---

20. Feature Transformation

Feature transformation changes the representation or distribution of a variable.

This can help models learn better.

Common transformations include:

- Log transformation
- Square-root transformation
- Power transformation
- Binning
- Polynomial transformation

Log Transformation

Suppose income is heavily right-skewed.

We can use:

```
import numpy as np

df["Income_log"] = np.log1p(df["Income"])
```

This can reduce extreme skewness.

Binning

Suppose we have:

```
Age
```

We can create:

```
18–25 → Young
26–40 → Adult
41–60 → Middle-aged
60+   → Senior
```

Example:

```
bins = [0, 25, 40, 60, 100]

labels = [
    "Young",
    "Adult",
    "Middle-aged",
    "Senior"
]

df["Age_Group"] = pd.cut(
    df["Age"],
    bins=bins,
    labels=labels
)
```

---

21. Feature Scaling

Different features can have very different ranges.

Example:

```
Age      → 18–80
Income   → 20,000–500,000
```

Some algorithms can be affected by these differences in scale.

Common scaling methods

```
Feature Scaling
│
├── Standardization
│
└── Normalization
```

Standardization

Transforms data approximately to:

```
Mean = 0
Standard deviation = 1
```

Using:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Min-Max Scaling

Transforms values into a specified range, commonly:

```
0 to 1
```

Using:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)
```

Important

Scaling is especially important for algorithms such as:

- KNN
- K-Means
- SVM
- Logistic Regression
- Linear Regression in many regularized settings
- Neural Networks

Tree-based models such as Decision Trees and Random Forests generally don't require feature scaling.

---

22. Feature Engineering

Definition

Feature engineering is the process of creating new useful features from existing data to improve model performance or represent the problem better.

This is one of the most valuable skills in ML.

Example 1 — Date

Original:

```
Date
2026-08-18
```

Create:

```
Year
Month
Day
Day_of_week
Weekend
```

Example 2 — Sales

Original:

```
Total Sales
Quantity
```

Create:

```
df["Price_Per_Unit"] = (
    df["Total Sales"] / df["Quantity"]
)
```

Example 3 — Customer Data

Original:

```
Total Spending
Number of Purchases
```

Create:

```
Average Purchase Value
```

Formula:

```
Average Purchase Value =
Total Spending / Number of Purchases
```

Feature engineering mindset

Ask:

> **"Can I combine existing information to create a variable that better represents the problem?"**

---

23. Feature Selection

Feature selection means selecting the **most useful features** for the model.

Suppose you have:

```
100 features
```

but only 20 are useful.

You may want to remove irrelevant or redundant features.

Benefits:

- Faster training
- Less complexity
- Reduced overfitting
- Easier interpretation
- Sometimes better performance

---

24. Feature Selection Methods

The major approaches are:

```
Feature Selection
│
├── Filter Methods
├── Wrapper Methods
└── Embedded Methods
```

You specifically need to understand **Filter** and **Embedded** methods.

---

25. Filter Method

Definition

Filter methods select features using statistical properties of the data, independently of the machine learning model.

The model is not trained repeatedly during selection.

Think:

```
Data
 ↓
Statistical Test
 ↓
Select Important Features
 ↓
ML Model
```

Common Filter Methods

1. Correlation

If two numerical features are highly correlated:

```
Feature A ↔ Feature B = 0.98
```

they may contain very similar information.

You may remove one of them depending on the problem and model.

---

2. Chi-Square Test

Useful for relationships between:

- Categorical features
- Categorical target

Example:

```
Gender ↔ Churn
```

---

3. ANOVA

Can be used to evaluate relationships between categorical groups and numerical targets/features under appropriate assumptions.

---

4. Mutual Information

Measures how much information one variable provides about another.

```
from sklearn.feature_selection import mutual_info_classif
```

Higher mutual information can indicate a stronger relationship with the target.

Advantages of Filter Methods

- Fast
- Simple
- Model-independent
- Good for datasets with many features

Disadvantages

- May ignore interactions between features
- A statistically useful feature isn't automatically useful for every model

---

26. Embedded Method

Definition

Embedded methods perform feature selection as part of the model-training process.

The model itself determines which features are important.

Conceptually:

```
Data
 ↓
ML Algorithm
 ↓
Feature Selection + Model Training
 ↓
Final Model
```

Common Embedded Methods

1. Lasso Regression

Lasso uses **L1 regularization**.

Some feature coefficients can become exactly zero.

Example:

```
Feature       Coefficient

Age             0.32
Income          0.75
Distance        0.00
Visits          0.41
```

`Distance` could therefore be removed from the model.

---

2. Decision Trees

Decision trees choose features based on how useful they are for splitting the data.

Example:

```
Feature Importance

Income       → 0.42
Age          → 0.25
Tenure       → 0.20
Gender       → 0.03
```

Features with very low importance may be candidates for removal.

---

3. Random Forest

Random Forest can provide feature importance estimates.

```
model.feature_importances_
```

---

27. Filter vs Embedded Methods

Filter Method	Embedded Method
Independent of model	Part of model training
Uses statistical measures	Uses model's learning process
Usually faster	Can be more computationally involved
Model-independent	Model-dependent
Correlation	Lasso
Chi-square	Decision Tree
Mutual Information	Random Forest

Easy memory trick

> **Filter → Select first, model later.**
>
> **Embedded → Selection happens while the model learns.**

---

28. Complete Data Preparation Pipeline

This is the workflow you should memorize for ML:

```
                RAW DATA
                   ↓
            ┌──────────────┐
            │     EDA      │
            └──────────────┘
                   ↓
```

```
        Understand Structure
                  ↓
        Missing Value Analysis
                  ↓
          Duplicate Detection
                  ↓
          Data Type Correction
                  ↓
        Category Standardization
                  ↓
         Logic/Domain Validation
                  ↓
            Outlier Analysis
                  ↓
          ┌─────────────────┐
          │ Data Cleaning   │
          └─────────────────┘
                  ↓
          Categorical Encoding
                  ↓
          Feature Transformation
                  ↓
             Feature Scaling
                  ↓
          Feature Engineering
                  ↓
            Feature Selection
           ↙      ↓      ↘
        Filter   Wrapper  Embedded
                  ↓
             ML MODEL
```

---

29. The Most Important Strategy

When you receive a new dataset, **don't immediately start cleaning everything**.

Use this sequence:

🔍 Ask

What is the problem?

↓

👀 Inspect

What does the data look like?

↓

🚨 Detect

What is wrong with it?

↓

🧹 Clean

How should I fix those problems?

↓

🔄 Transform

What format does the model need?

↓

🛠️ Engineer

Can I create better features?

↓

🎯 Select

Which features are actually useful?

↓

🤖 Model

Now train the ML algorithm.

---

30. EDA vs Data Cleaning vs Preprocessing

These terms are related but different.

EDA

Understand the data.

```
"What is happening?"
```

Data Cleaning

Fix problems in the data.

```
"What is wrong and how do I fix it?"
```

Data Preprocessing

Convert the cleaned data into a format suitable for ML.

```
"How do I prepare it for the algorithm?"
```

Feature Engineering

Create better representations of the problem.

```
"Can I create more useful information?"
```

Feature Selection

Keep the most useful information.

```
"Which features should the model actually use?"
```

🧠 Quick Revision Cheat Sheet

```
EDA
→ Understand the data

View Data
→ head(), tail(), sample(), shape, info()
```

```
Statistics
→ describe()

Value Counts
→ value_counts()

Missing Values
→ isnull()

Visualization
→ histplot(), boxplot(), scatterplot(), heatmap()

Target Exploration
→ Understand distribution of y

Data Cleaning
→ Fix problems

Duplicates
→ duplicated(), drop_duplicates()

Data Types
→ astype(), to_numeric(), to_datetime()

Inconsistent Categories
→ strip(), lower(), replace()

Outliers
→ Boxplot, IQR, Z-score

Logic Errors
→ Values that violate logical relationships

Domain Errors
→ Values outside valid ranges

Preprocessing
→ Prepare data for ML

Encoding
→ Convert categories to numbers

Transformation
→ Change feature representation/distribution

Scaling
→ Put numerical features on comparable scales
```

```
Feature Engineering
→ Create useful new features

Feature Selection
→ Keep useful features

Filter
→ Statistical selection before modeling

Embedded
→ Selection during model training
```

🚀 What You Should Practice

Don't just memorize these definitions.

Take one dataset and perform the entire pipeline:

```
Customer Churn Dataset
        ↓
EDA
        ↓
Missing values
        ↓
Duplicates
        ↓
Data types
        ↓
Inconsistent categories
        ↓
Outliers
        ↓
Target exploration
        ↓
Encoding
        ↓
Transformation
        ↓
Scaling
        ↓
Feature engineering
        ↓
```

```
Feature selection
        ↓
Ready for ML
```

That single exercise will teach you far more than reading ten separate tutorials.

Next ML topic

After this, move to:

Train/Test Split → Overfitting → Underfitting → Bias/Variance → Cross-Validation → Data Leakage

These are the concepts you need **before training your first serious ML model**.